

Лабораторная работа №4

Компьютерный практикум по статистическому анализу

Надежда Александровна Рогожина

2025-10-21

Содержание I

1. Информация

2. Введение

3. Выполнение лабораторной работы

4. Выводы

Раздел 1

1. Информация

1.1 Докладчик

► Рогожина Надежда Александровна

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22
- ▶ Российский университет дружбы народов им. П. Лумумбы

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22
- ▶ Российский университет дружбы народов им. П. Лумумбы
- ▶ 1132222840@rudn.ru

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22
- ▶ Российский университет дружбы народов им. П. Лумумбы
- ▶ 1132222840@rudn.ru
- ▶ <https://MikoGreen.github.io/ru/>

Раздел 2

2. Введение

2.1 Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

2.2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 4.2.

2.2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 4.2.
2. Выполните задания для самостоятельной работы (раздел 4.4).

2.3 Теоретическое введение

Научные вычисления традиционно требуют высочайшей производительности, однако эксперты по доменам в значительной степени перенесены на более медленные динамические языки для ежедневной работы. К счастью, методы современного языка и компилятора позволяют в основном устранить компромисс производительности и обеспечивать достаточно продуктивную среду для прототипирования и достаточно эффективно для развертывания применений, интенсивных производительности. Язык программирования Julia заполняет эту роль: это гибкий динамический язык, подходящий для научных и численных вычислений, с производительностью, сравнимой с традиционными статичными языками.

Раздел 3

3. Выполнение лабораторной работы

3.1 Примеры

Позлементные операции над многомерными массивами

```
In [1]: a = rand(1:20,(4,3))
```

```
Out[1]: 4x3 Matrix{Int64}:  
  2  4 18  
 14  1  5  
  6  2 14  
  1 12 18
```

```
In [2]: # Позлементная сумма:  
sum(a)
```

```
Out[2]: 97
```

```
In [3]: # Позлементная сумма по столбцам:  
sum(a,dims=1)
```

```
Out[3]: 1x3 Matrix{Int64}:  
 23 19 55
```

```
In [4]: # Позлементная сумма по строкам:  
sum(a,dims=2)
```

```
Out[4]: 4x1 Matrix{Int64}:  
 24  
 20  
 22  
 31
```

Рисунок 1: Массивы

3.2 Примеры

```
In [5]: # Поэлементное произведение:  
prod(a)
```

```
Out[5]: 365783040
```

```
In [6]: # Поэлементное произведение по столбцам:  
prod(a,dims=1)
```

```
Out[6]: 1x3 Matrix{Int64}:  
168  96 22680
```

```
In [8]: # Поэлементное произведение по строкам:  
prod(a,dims=2)
```

```
Out[8]: 4x1 Matrix{Int64}:  
144  
70  
168  
216
```

Рисунок 2: Массивы

3.3 Примеры

```
In [10]: # Вычисление среднего значения массива:  
mean(a)
```

```
Out[10]: 8.083333333333334
```

```
In [11]: # Среднее по столбцам:  
mean(a,dims=1)
```

```
Out[11]: 1x3 Matrix{Float64}:  
 5.75  4.75 13.75
```

```
In [12]: # Среднее по строкам:  
mean(a,dims=2)
```

```
Out[12]: 4x1 Matrix{Float64}:  
 8.0  
 6.666666666666667  
 7.333333333333333  
10.333333333333334
```

Рисунок 3: Массивы

3.4 Примеры

Транспонирование, след, ранг, определитель и инверсия матрицы

```
In [14]: # Массив 4x4 со случайными целыми числами (от 1 до 20):  
b = rand(1:20,(4,4))
```

```
Out[14]: 4x4 Matrix{Int64}:  
13 14 16 14  
2 20 12 13  
10 4 11 3  
13 8 6 9
```

```
In [15]: # Транспонирование:  
transpose(b)
```

```
Out[15]: 4x4 transpose(::Matrix{Int64}) with eltype Int64:  
13 2 10 13  
14 20 4 8  
16 12 11 6  
14 13 3 9
```

```
In [16]: # След матрицы (сумма диагональных элементов):  
tr(b)
```

```
Out[16]: 53
```

Рисунок 4: Матрицы

3.5 Примеры

```
In [17]: # Извлечение диагональных элементов как массив:  
diag(b)
```

```
Out[17]: 4-element Vector{Int64}:  
 13  
 20  
 11  
 9
```

```
In [18]: # Ранг матрицы:  
rank(b)
```

```
Out[18]: 4
```

```
In [19]: # Инверсия матрицы (определение обратной матрицы):  
inv(b)
```

```
Out[19]: 4x4 Matrix{Float64}:  
-0.0992908  0.0070922  0.070922  0.120567  
-0.235739  0.142615  0.121801  0.120105  
 0.0974406 -0.0255936  0.048412  -0.130743  
 0.288005  -0.119951  -0.242985  -0.0826395
```

Рисунок 5: Матрицы

3.6 Примеры

```
In [20]: # Определитель матрицы:  
det(b)
```

```
Out[20]: -6486.000000000001
```

```
In [21]: # Псевдобратная функция для прямоугольных матриц:  
pinv(a)
```

```
Out[21]: 3x4 Matrix{Float64}:  
-0.0243961  0.0734136  0.00325503  0.00147172  
-0.0732617  0.0305137 -0.0661507   0.116236  
0.0536149  -0.0230897  0.0397668  -0.0225753
```

Рисунок 6: Матрицы

3.7 Примеры

Вычисление нормы векторов и матриц, повороты, вращения

```
In [22]: # Создание вектора X:  
X = [2, 4, -5]
```

```
Out[22]: 3-element Vector{Int64}:  
 2  
 4  
-5
```

```
In [23]: # Вычисление евклидовой нормы:  
norm(X)
```

```
Out[23]: 6.708203932499369
```

```
In [30]: # Вычисление p-нормы:  
p = 1  
norm(X,p)
```

```
Out[30]: 11.0
```

Рисунок 7: Норма, поворот, вращение

3.8 Примеры

```
In [25]: # Расстояние между двумя векторами X и Y:  
X = [2, 4, -5];  
Y = [1, -1, 3];  
norm(X-Y)
```

```
Out[25]: 9.486832980505138
```

```
In [26]: # Проверка по базовому определению:  
sqrt(sum((X-Y).^2))
```

```
Out[26]: 9.486832980505138
```

```
In [31]: # Угол между двумя векторами:  
acos((transpose(X)*Y)/(norm(X)*norm(Y)))
```

```
Out[31]: 2.4404307889469252
```

Рисунок 8: Норма, поворот, вращение

3.9 Примеры

```
In [32]: # Создание матрицы:  
d = [5 -4 2 ; -1 2 3; -2 1 0]
```

```
Out[32]: 3x3 Matrix{Int64}:  
  5  -4  2  
 -1   2  3  
 -2   1  0
```

```
In [33]: # Вычисление Евклидовой нормы:  
opnorm(d)
```

```
Out[33]: 7.147682841795258
```

```
In [34]: # Вычисление p-нормы:  
p=1  
opnorm(d,p)
```

```
Out[34]: 8.0
```

Рисунок 9: Норма, поворот, вращение

3.10 Примеры

```
In [35]: # Поворот на 180 градусов:  
rot180(d)
```

```
Out[35]: 3x3 Matrix{Int64}:  
  0  1 -2  
  3  2 -1  
  2 -4  5
```

```
In [36]: # Переворачивание строк:  
reverse(d,dims=1)
```

```
Out[36]: 3x3 Matrix{Int64}:  
 -2  1  0  
 -1  2  3  
  5 -4  2
```

```
In [37]: # Переворачивание столбцов  
reverse(d,dims=2)
```

```
Out[37]: 3x3 Matrix{Int64}:  
  2 -4  5  
  3  2 -1  
  0  1 -2
```

Рисунок 10: Норма, поворот, вращение

3.11 Примеры

Матричное умножение, единичная матрица, скалярное произведение

```
In [38]: # Матрица 2x3 со случайными целыми значениями от 1 до 10:  
A = rand(1:10,(2,3))  
# Матрица 3x4 со случайными целыми значениями от 1 до 10:  
B = rand(1:10,(3,4))
```

```
Out[38]: 3x4 Matrix{Int64}:  
 4  8  2  2  
 7 10  8 10  
 7  9  2  4
```

```
In [39]: # Произведение матриц A и B:  
A*B
```

```
Out[39]: 2x4 Matrix{Int64}:  
145 222 92 122  
 74 122 52  64
```

Рисунок 11: Матрицы

3.12 Примеры

```
In [40]: # Единичная матрица 3x3:  
Matrix{Int}(I, 3, 3)
```

```
Out[40]: 3x3 Matrix{Int64}:  
 1  0  0  
 0  1  0  
 0  0  1
```

```
In [41]: # Скалярное произведение векторов X и Y:  
X = [2, 4, -5]  
Y = [1, -1, 3]  
dot(X, Y)
```

```
Out[41]: -17
```

```
In [42]: # тоже скалярное произведение:  
X'Y
```

```
Out[42]: -17
```

Рисунок 12: Матрицы

3.13 Примеры

Факторизация. Специальные матричные структуры

```
In [43]: # Задаём квадратную матрицу 3x3 со случайными значениями:  
A = rand(3, 3)
```

```
Out[43]: 3x3 Matrix{Float64}:  
 0.810242  0.49526  0.494848  
 0.979296  0.575608  0.164226  
 0.303989  0.285174  0.526203
```

```
In [44]: # Задаём единичный вектор:  
x = fill(1.0, 3)
```

```
Out[44]: 3-element Vector{Float64}:  
 1.0  
 1.0  
 1.0
```

```
In [45]: # Задаём вектор b:  
b = A*x
```

```
Out[45]: 3-element Vector{Float64}:  
 1.8003491505283473  
 1.7191301016635443  
 1.1153657883881851
```

```
In [46]: A\b
```

```
Out[46]: 3-element Vector{Float64}:  
 1.0000000000000009  
 0.9999999999999986  
 1.0000000000000007
```

Рисунок 13: Факторизация

3.14 Примеры

```
In [47]: A1u = lu(A)

Out[47]: LU{Float64, Matrix{Float64}, Vector{Int64}}
L factor:
3x3 Matrix{Float64}:
 1.0      0.0      0.0
 0.310416 1.0      0.0
 0.827372 0.178571 1.0
U factor:
3x3 Matrix{Float64}:
 0.979296 0.575608 0.164226
 0.0      0.106496 0.475225
 0.0      0.0      0.27411
```

```
In [82]: # Матрица перестановок:
A1u.P
```

```
Out[82]: 3x3 Matrix{Float64}:
 0.0  1.0  0.0
 0.0  0.0  1.0
 1.0  0.0  0.0
```

```
In [49]: # Вектор перестановок:
A1u.p
```

```
Out[49]: 3-element Vector{Int64}:
 2
 3
 1
```

Рисунок 14: Факторизация

3.15 Примеры

```
In [50]: # Матрица L:  
Alu.L
```

```
Out[50]: 3x3 Matrix{Float64}:  
 1.0      0.0      0.0  
 0.310416 1.0      0.0  
 0.827372 0.178571 1.0
```

```
In [51]: # Матрица U:  
Alu.U
```

```
Out[51]: 3x3 Matrix{Float64}:  
 0.979296 0.575608 0.164226  
 0.0      0.106496 0.475225  
 0.0      0.0      0.27411
```

Рисунок 15: Факторизация

3.16 Примеры

```
In [52]: # Решение СЛАУ через матрицу A:  
A\b
```

```
Out[52]: 3-element Vector{Float64}:  
 1.0000000000000009  
 0.9999999999999986  
 1.0000000000000007
```

```
In [53]: # Решение СЛАУ через объект факторизации:  
Alu\b
```

```
Out[53]: 3-element Vector{Float64}:  
 1.0000000000000009  
 0.9999999999999986  
 1.0000000000000007
```

Рисунок 16: Факторизация

3.17 Примеры

```
In [54]: # Детерминант матрицы A:  
det(A)
```

```
Out[54]: 0.02858723726749104
```

```
In [55]: # Детерминант матрицы A через объект факторизации:  
det(Alu)
```

```
Out[55]: 0.02858723726749104
```

Рисунок 17: Факторизация

3.18 Примеры

```
In [56]: # QR-факторизация:
Aqr = qr(A)

Out[56]: LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
Q factor: 3x3 LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
R factor:
3x3 Matrix{Float64}:
-1.30687  -0.804714  -0.552258
  0.0      0.101815   0.444507
  0.0      0.0        -0.214846

In [57]: # Матрица Q:
Aqr.Q

Out[57]: 3x3 LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}

In [58]: # Матрица R:
Aqr.R

Out[58]: 3x3 Matrix{Float64}:
-1.30687  -0.804714  -0.552258
  0.0      0.101815   0.444507
  0.0      0.0        -0.214846
```

Рисунок 18: Факторизация

3.19 Примеры

```
In [59]: # Проверка, что матрица Q - ортогональная:  
Aqr.Q**Aqr.Q
```

```
Out[59]: 3x3 Matrix{Float64}:  
 1.0      6.93889e-18  1.11022e-16  
 0.0      1.0      -1.11022e-16  
 2.77556e-17 -1.11022e-16  1.0
```

```
In [60]: # Симметризация матрицы A:  
Asym = A + A'
```

```
Out[60]: 3x3 Matrix{Float64}:  
 1.62048  1.47456  0.798836  
 1.47456  1.15122  0.4494  
 0.798836  0.4494  1.05241
```

Рисунок 19: Факторизация

3.20 Примеры

```
In [61]: # Спектральное разложение симметризованной матрицы:  
AsymEig = eigen(Asym)
```

```
Out[61]: Eigen(Float64, Float64, Matrix{Float64}, Vector{Float64})  
values:  
3-element Vector{Float64}:  
 -0.14100920389628646  
  0.7163245396813496  
  3.248791833981368  
vectors:  
3x3 Matrix{Float64}:  
  0.679083  -0.160315  -0.716341  
 -0.709768  -0.392374  -0.58504  
 -0.187283   0.905727  -0.380241
```

```
In [63]: # Собственные значения:  
AsymEig.values
```

```
Out[63]: 3-element Vector{Float64}:  
 -0.14100920389628646  
  0.7163245396813496  
  3.248791833981368
```

Рисунок 20: Факторизация

3.21 Примеры

```
In [64]: #Собственные векторы:  
AsymEig.vectors
```

```
Out[64]: 3x3 Matrix{Float64}:  
  0.679083  -0.160315  -0.716341  
 -0.709768  -0.392374  -0.58504  
 -0.187283   0.905727  -0.380241
```

```
In [65]: # Проверяем, что получится единичная матрица:  
inv(AsymEig)*Asym
```

```
Out[65]: 3x3 Matrix{Float64}:  
  1.0          -3.83027e-15  -2.10942e-15  
 -7.54952e-15   1.0          3.9968e-15  
 -5.55112e-16   1.72085e-15   1.0
```

Рисунок 21: Факторизация

3.22 Примеры

```

In [68]: # Симметризация матрицы:
        Asym = A + A'

Out[68]: 1000x1000 Matrix{Float64}:
        1.40201 -1.39007 -1.35224 -0.517401 -0.478665 -0.619091
        -1.39007 -2.36956 -1.04276 1.59158 1.87155 -1.04341
        -1.35224 -1.04276 -3.68485 0.444404 -1.73212 0.0571082
        0.002853 1.26678 0.565504 1.23892 0.812473 1.032
        3.11771 1.07171 1.6957 0.137295 -1.71849 -2.21745
        0.116925 1.76072 -1.54503 0.152779 1.16923 -0.398563
        0.518404 -1.03108 -0.00338497 1.5379 -0.606369 -1.21191
        1.37983 1.22685 0.525823 0.76792 -3.3191 -0.555363
        -1.43576 -0.241476 0.675181 -1.72746 1.50828 -0.446107
        1.77932 1.8412 1.26364 -0.124385 0.653995 0.482288
        1.9551 -0.840306 -0.156152 -0.978462 -1.14105 -1.20475
        -0.473217 -2.18959 -0.204358 1.07812 -0.8872 2.56728
        1.15581 -0.261902 -2.0767 -0.400608 -1.09542 3.60179
        ⋮
        1.22649 -0.224395 -1.7631 -0.228362 0.290314 -0.471325
        -0.508161 -0.583138 0.17556 -2.77026 0.0615006 -1.51562
        1.95531 1.46106 -2.7824 -2.27826 -0.18502 -1.57939
        1.75525 -0.941328 -1.70585 -2.13705 1.83737 -0.66546
        0.968279 0.04188 -0.105008 0.677229 0.0205377 -1.7063
        2.82367 0.232324 -1.7645 -0.178778 0.43338 1.50674
        -1.28909 -0.376543 -0.092054 1.85804 -0.715886 2.75529
        1.26614 1.00836 0.180804 -1.83211 0.62033 0.044405
        -0.789682 -0.685007 -0.845038 0.533795 -1.22225 0.558872
        -0.517401 1.59158 0.444404 1.20111 -0.160947 -1.15042
        -0.478665 1.87155 -1.73212 -0.160947 -3.26246 3.77296
        -0.619091 -1.04341 0.0571082 -1.15042 3.77296 -1.06133

In [69]: # Проверка, является ли матрица симметричной:
        issymmetric(Asym)

Out[69]: true

```

Рисунок 22: Факторизация

3.23 Примеры

```
In [70]: # Добавление шума:  
Asym_noisy = copy(Asym)  
Asym_noisy[1,2] += 5eps()
```

```
Out[70]: -1.3900747021164384
```

```
In [71]: # Проверка, является ли матрица симметричной:  
issymmetric(Asym_noisy)
```

```
Out[71]: false
```

Рисунок 23: Факторизация

3.24 Примеры

```
In [72]: # Явно указываем, что матрица является симметричной:
        Asym_explicit = Symmetric(Asym_noisy)

Out[72]: 1000x1000 Symmetric(Float64, Matrix{Float64}):
        1.40201  -1.39007  -1.35224  ...  -0.517401  -0.478665  -0.619091
        -1.39007  -2.36956  -1.04276  ...  1.59158  1.87155  -1.04341
        -1.35224  -1.04276  -3.68485  ...  0.444404  -1.73212  0.0571082
        0.002853  1.26678  0.565504  ...  1.23892  0.812473  1.032
        3.11771  1.07171  1.6957  ...  0.137295  -1.71849  -2.21745
        0.116925  1.76072  -1.54503  ...  0.152779  1.16923  -0.398563
        0.518404  -1.03108  -0.00338497  ...  1.5379  -0.606369  -1.21191
        1.37983  1.22685  0.525823  ...  0.76792  -3.3191  -0.555363
        -1.43576  -0.241476  0.675181  ...  -1.72746  1.50828  -0.446107
        1.77932  1.8412  1.26364  ...  -0.124385  0.653995  0.482288
        1.9551  -0.840306  -0.156152  ...  -0.978462  -1.14105  -1.20475
        -0.473217  -2.18959  -0.204358  ...  1.07812  -0.8872  2.56728
        1.15581  -0.261902  -2.0767  ...  -0.400608  -1.09542  3.60179
        ⋮
        1.22649  -0.224395  -1.7631  ...  -0.228362  0.290314  -0.471325
        -0.508161  -0.583138  0.17556  ...  -2.77026  0.0615006  -1.51562
        1.95531  1.46106  -2.7824  ...  -2.27826  -0.18502  -1.57939
        1.75525  -0.941328  -1.70585  ...  -2.13705  1.83737  -0.66546
        0.968279  0.04188  -0.105008  ...  0.677229  0.0205377  -1.7063
        2.82367  0.232324  -1.7645  ...  -0.178778  0.43338  1.50674
        -1.28909  -0.376543  -0.092054  ...  1.85804  -0.715886  2.75529
        1.26614  1.00836  0.180804  ...  -1.83211  0.62033  0.044405
        -0.789682  -0.685007  -0.845038  ...  0.533795  -1.22225  0.558872
        -0.517401  1.59158  0.444404  ...  1.20111  -0.160947  -1.15042
        -0.478665  1.87155  -1.73212  ...  -0.160947  -3.26246  3.77296
        -0.619091  -1.04341  0.0571082  ...  -1.15042  3.77296  -1.06133

In [74]: # Проверка, является ли матрица симметричной:
        issymmetric(Asym_explicit)

Out[74]: true
```

Рисунок 24: Факторизация

3.25 Примеры

```
In [75]: import Pkg
        Pkg.add("BenchmarkTools")
        using BenchmarkTools

        Resolving package versions...
        Installed BenchmarkTools = v1.6.2
        Updating `C:\Users\Home\.julia\environments\v1.11\Project.toml`
        [6e4b80f9] + BenchmarkTools v1.6.2
        Updating `C:\Users\Home\.julia\environments\v1.11\Manifest.toml`
        [6e4b80f9] + BenchmarkTools v1.6.2
        [9abbd945] + Profile v1.11.0
        Precompiling project...
        4747.0 ms ✓ BenchmarkTools
        1 dependency successfully precompiled in 9 seconds. 481 already precompiled.
```

```
In [76]: # Оценка эффективности выполнения операции по нахождению
        # собственных значений симметризованной матрицы:
        @btime eigvals(Asym);

        86.969 ms (21 allocations: 7.99 MiB)
```

```
In [77]: # Оценка эффективности выполнения операции по нахождению
        # собственных значений зашумленной матрицы:
        @btime eigvals(Asym_noisy);

        597.426 ms (27 allocations: 7.93 MiB)
```

Рисунок 25: Факторизация

3.26 Примеры

Out[80]: 6.445330226185416

Рисунок 26: Факторизация

3.27 Примеры

Общая линейная алгебра

```
In [83]: # Матрица с рациональными элементами:  
Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3))/10
```

```
Out[83]: 3x3 Matrix{Rational{BigInt}}:  
 9//10  1      4//5  
 3//10  3//5   1  
 1//10  7//10  1//10
```

```
In [84]: # Единичный вектор:  
x = fill(1, 3)
```

```
Out[84]: 3-element Vector{Int64}:  
 1  
 1  
 1
```

```
In [85]: # Задаём вектор b:  
b = Arational*x
```

```
Out[85]: 3-element Vector{Rational{BigInt}}:  
 27//10  
 19//10  
 9//10
```

Рисунок 27: Линейная алгебра

3.28 Примеры

```
In [86]: # Решение исходного уравнения получаем с помощью функции \
# (убеждаемся, что x - единичный вектор):
Arational\b
```

```
Out[86]: 3-element Vector{Rational{BigInt}}:
 1
 1
 1
```

```
In [87]: # LU-разложение:
lu(Arational)
```

```
Out[87]: LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64}}
L factor:
3x3 Matrix{Rational{BigInt}}:
 1      0      0
1//9    1      0
1//3 24//53    1
U factor:
3x3 Matrix{Rational{BigInt}}:
9//10    1      4//5
 0      53//90    1//90
 0      0      193//265
```

Рисунок 28: Линейная алгебра

3.29 Задачи для самостоятельного выполнения

Далее, необходимо было выполнить *Задания для самостоятельного выполнения*:

3.30 Произведение векторов

Задания для самостоятельного выполнения

Произведение векторов

```
In [90]: v = [2, 4, 8]
```

```
Out[90]: 3-element Vector{Int64}:  
 2  
 4  
 8
```

```
In [91]: dot_v = dot(v,v)
```

```
Out[91]: 84
```

```
In [96]: outer_v = v*v'
```

```
Out[96]: 3x3 Matrix{Int64}:  
 4  8 16  
 8 16 32  
16 32 64
```

Рисунок 29: Векторы

3.31 Системы линейных уравнений

Системы линейных уравнений

```
In [322]: function solve_LS(A, b)
if size(A)[1] == size(A)[2]
    det_A = det(A)
    println("det(A) = $det_A")
    if det_A != 0
        prob = LS.LinearProblem(A, b)
        sol = LS.solve(prob)
        if size(A)[2] == 2
            x, y = sol.u
            println("calculation LS: x = $x, y = $y")
        elseif size(A)[2] == 3
            x, y, z = sol.u
            println("calculation LS: x = $x, y = $y, z = $z")
        end
    end
    ALU = lu(A)
    sol2 = ALU\b
    if size(A)[2] == 2
        x, y = sol2
        println("calculation lu: x = $x, y = $y")
    elseif size(A)[2] == 3
        x, y, z = sol2
        println("calculation lu: x = $x, y = $y, z = $z")
    end
else
    println("Определитель = 0, матрица вырожденная")
end
else
    sol2 = A\b
    if size(A)[2] == 2
        x, y = sol2
        println("calculation A\b: x = $x, y = $y")
    elseif size(A)[2] == 3
        x, y, z = sol2
        println("calculation A\b: x = $x, y = $y, z = $z")
    end
end
end
```

Out[322]: solve_LS (generic function with 1 method)

Рисунок 30: Функция

3.32 Системы линейных уравнений

```
In [323]: A = (Float64)[[1, 1] [1, -1]]  
          b = (Float64)[2, 3]  
          solve_LS(A, b)
```

```
det(A) = -2.0  
calculation LS: x = 2.5, y = -0.5  
calculation lu: x = 2.5, y = -0.5
```

```
In [324]: A = (Float64)[[1, 2] [1, 2]]  
          b = (Float64)[2, 4]  
          solve_LS(A, b)
```

```
det(A) = 0.0  
Определитель = 0, матрица вырожденная
```

```
In [325]: A = (Float64)[[1, 2] [1, 2]]  
          b = (Float64)[2, 5]  
          solve_LS(A, b)
```

```
det(A) = 0.0  
Определитель = 0, матрица вырожденная
```

Рисунок 31: СЛАУ с двумя неизвестными

3.33 Системы линейных уравнений

```
In [326]: A = (Float64)[[1, 2, 3] [1, 2, 3]]  
          b = (Float64)[1, 2, 3]  
          solve_LS(A, b)
```

```
calculation : x = 0.5, y = 0.5
```

```
In [327]: A = (Float64)[[1, 2, 1] [1, 1, -1]]  
          b = (Float64)[2, 1, 3]  
          solve_LS(A, b)
```

```
calculation : x = 1.5000000000000004, y = -0.9999999999999997
```

```
In [328]: A = (Float64)[[1, 2, 3] [1, 1, 2]]  
          b = (Float64)[2, 1, 3]  
          solve_LS(A, b)
```

```
calculation : x = -0.9999999999999994, y = 2.999999999999999
```

Рисунок 32: СЛАУ с двумя неизвестными

3.34 Системы линейных уравнений

```
In [332]: A = (Float64)[[1, 1] [1, -1] [1, -2]]  
          b = (Float64)[2, 3]  
          solve_LS(A, b)  
  
calculation : x = 2.214285714285715, y = 0.35714285714285715, z = -0.5714285714285711
```

```
In [330]: A = (Float64)[[1, 2, 3] [1, 2, 1] [1, -3, 1]]  
          b = (Float64)[2, 4, 1]  
          solve_LS(A, b)  
  
det(A) = -10.0  
calculation LS: x = -0.5, y = 2.5, z = 0.0  
calculation LU: x = -0.5, y = 2.5, z = 0.0
```

```
In [331]: A = (Float64)[[1, 1, 2] [1, 1, 2] [1, 2, 3]]  
          b = (Float64)[1, 0, 1]  
          solve_LS(A, b)  
  
det(A) = 0.0  
Определитель = 0, матрица вырожденная
```

```
In [333]: A = (Float64)[[1, 1, 2] [1, 1, 2] [1, 2, 3]]  
          b = (Float64)[1, 0, 0]  
          solve_LS(A, b)  
  
det(A) = 0.0  
Определитель = 0, матрица вырожденная
```

Рисунок 33: СЛАУ с тремя неизвестными

3.35 Операции с матрицами

Операции с матрицами

```
In [340]: A = (Float64)[[1, -2] [-2, 1]]  
          Diagonal(A)
```

```
Out[340]: 2x2 Diagonal{Float64, Vector{Float64}}:  
 1.0  .  
  .  1.0
```

```
In [339]: A = (Float64)[[1, -2] [-2, 3]]  
          Diagonal(A)
```

```
Out[339]: 2x2 Diagonal{Float64, Vector{Float64}}:  
 1.0  .  
  .  3.0
```

```
In [341]: A = (Float64)[[1, -2, 0] [-2, 1, 2] [0, 2, 0]]  
          Diagonal(A)
```

```
Out[341]: 3x3 Diagonal{Float64, Vector{Float64}}:  
 1.0  .  .  
  .  1.0  .  
  .  .  0.0
```

Рисунок 34: Диагональный вид

3.36 Операции с матрицами

```
In [342]: A = (Float64)[[1, -2] [-2, 1]]  
A.^ 10
```

```
Out[342]: 2x2 Matrix{Float64}:  
 1.0 1024.0  
1024.0 1.0
```

```
In [346]: A = (Float64)[[5, -2] [-2, 5]]  
A ^ (1//2)
```

```
Out[346]: 2x2 Symmetric{Float64, Matrix{Float64}}:  
 2.1889 -0.45685  
-0.45685 2.1889
```

```
In [347]: A = (Float64)[[5, -2] [-2, 5]]  
A ^ (1//3)
```

```
Out[347]: 2x2 Symmetric{Float64, Matrix{Float64}}:  
 1.67759 -0.235341  
-0.235341 1.67759
```

Рисунок 35: Решение

3.37 Операции с матрицами

```
In [351]: A = (Float64)[[1, 2] [2, 3]]  
          A = (3//2)  
  
Out[351]: 2x2 Symmetric{ComplexF64, Matrix{ComplexF64}}:  
           0.568864+0.351578im  0.928442-0.217287im  
           0.928442-0.217287im   1.48931+0.134291im  
  
In [352]: A = [[140, 97, 74, 168, 131] [97, 106, 89, 131, 36] [74, 80, 152, 144, 71] [168, 131, 144, 54, 142] [131, 36, 71, 142, 36]]  
  
Out[352]: 5x5 Matrix{Int64}:  
           140  97  74 168 131  
           97 106  89 131  36  
           74  89 152 144  71  
           168 131 144  54 142  
           131  36  71 142  36
```

Рисунок 36: Решение

3.38 Операции с матрицами

```
In [362]: @btime Diagonal(eigen(A).values)
          4.343 μs (27 allocations: 6.53 KiB)

Out[362]: 5x5 Diagonal{Float64, Vector{Float64}}:
 36.0  .  .  .  .
 .  54.0  .  .  .
 .  .  106.0  .  .
 .  .  .  140.0  .
 .  .  .  .  152.0

In [361]: @btime trill(A)
          73.837 ns (0 allocations: 0 bytes)

Out[361]: 5x5 Matrix{Int64}:
140  0  0  0  0
 97 106 0  0  0
 74  89 152 0  0
168 131 144 54  0
131  36  71 142 36
```

Рисунок 37: Работа с матрицей A

3.39 Линейные модели экономики

Линейные модели экономики

```
In [389]: A = (Float64)[[1, 3] [2, 4]]  
          b = (Float64)[1, 1]  
          solve_LS(A, b)
```

```
det(A) = -2.0  
calculation LS: x = -1.0, y = 1.0  
calculation lu: x = -1.0, y = 1.0
```

```
In [388]: A = 0.5 * (Float64)[[1, 3] [2, 4]]  
          b = (Float64)[1, 1]  
          solve_LS(A, b)
```

```
det(A) = -0.5  
calculation LS: x = -2.0, y = 2.0  
calculation lu: x = -2.0, y = 2.0
```

```
In [390]: A = 0.1 * (Float64)[[1, 3] [2, 4]]  
          b = (Float64)[1, 1]  
          solve_LS(A, b)
```

```
det(A) = -0.020000000000000007  
calculation LS: x = -9.999999999999998, y = 10.0  
calculation lu: x = -9.999999999999998, y = 10.0
```

Рисунок 38: Проверка

3.40 Линейные модели экономики

```
In [402]: A = (Float64){[1, 3] [2, 1]}  
          E = (Float64){[1, 0] [0, 1]}  
          inv(E-A) .>= 0
```

```
Out[402]: 2x2 BitMatrix:  
 1  0  
 0  1
```

```
In [403]: A = 0.5 * (Float64){[1, 3] [2, 1]}  
          E = (Float64){[1, 0] [0, 1]}  
          inv(E-A) .>= 0
```

```
Out[403]: 2x2 BitMatrix:  
 0  0  
 0  0
```

```
In [404]: A = 0.1 * (Float64){[1, 3] [2, 1]}  
          E = (Float64){[1, 0] [0, 1]}  
          inv(E-A) .>= 0
```

```
Out[404]: 2x2 BitMatrix:  
 1  1  
 1  1
```

Рисунок 39: Проверка

3.41 Линейные модели экономики

```
In [411]: A = (Float64)[[1, 3] [2, 1]]
          abs.(eigen(A).values) .< 1

Out[411]: 2-element BitVector:
           0
           0

In [412]: A = 0.5 * (Float64)[[1, 3] [2, 1]]
          abs.(eigen(A).values) .< 1

Out[412]: 2-element BitVector:
           1
           0

In [413]: A = 0.1 * (Float64)[[1, 3] [2, 1]]
          abs.(eigen(A).values) .< 1

Out[413]: 2-element BitVector:
           1
           1

In [414]: A = (Float64)[[0.1, 0, 0] [0.2, 0.1, 0.1] [0.3, 0.2, 0.3]]
          abs.(eigen(A).values) .< 1

Out[414]: 3-element BitVector:
           1
           1
           1
```

Рисунок 40: Задание 9

Раздел 4

4. Выводы

4.1 Выводы

В ходе работы были изучены основные способы решения задач линейной алгебры в я.п. Julia.